 Run in Google Colab  View on Github  View raw on Github

Module 13: Texts

```
!pip install wordcloud spacy
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: numpy>=1.6.1 in /usr/local/lib/python3.11/dis
Requirement already satisfied: pillow in /usr/local/lib/python3.11/dist-pack
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/p
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/pytho
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/pyt
Requirement already satisfied: weasel<0.5.0,>=0.1.0 in /usr/local/lib/python
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/pyt
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-pack
Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/
Requirement already satisfied: langcodes<4.0.0,>=3.2.0 in /usr/local/lib/pyt
Requirement already satisfied: language-data>=1.2 in /usr/local/lib/python3.
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/pyth
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/pytho
Requirement already satisfied: typing-extensions>=4.12.2 in /usr/local/lib/p
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/py
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/py
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dis
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/py
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.11/dis
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.11/di
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/
```

```
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/py
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dis
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.1
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.1
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python
Requirement already satisfied: marisa-trie>=1.1.0 in /usr/local/lib/python3.
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-pa
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/pytho
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/pyt
Requirement already satisfied: wrapt in /usr/local/lib/python3.11/dist-packa
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 94.0 MB/s eta 0:0
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting t
'Restart kernel' or 'Restart runtime' option.
```

We'll use `spaCy` and `wordcloud` to play with text data. `spaCy` is probably the best python package for analyzing text data. It's capable and super fast. Let's install them.

```
pip install wordcloud spacy
```

To use spaCy, you also need to download models. Run:

```
python -m spacy download en_core_web_sm
```

SpaCy basics

```
import spacy
import wordcloud
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from collections import Counter

# Load the spaCy language model
nlp = spacy.load('en_core_web_sm')
```

Usually the first step of text analysis is *tokenization*, which is the process of breaking a document into "tokens". You can roughly think of it as extracting each word.

```
doc = nlp(u'Apple is looking at buying U.K. startup for $1 billion')

for token in doc:
    print(token)
```

```
Apple
is
looking
at
buying
U.K.
startup
for
$
1
billion
```

As you can see, it's not exactly same as `doc.split()`. You'd want to have `$` as a separate token because it has a particular meaning (USD). Actually, as shown in an example (https://spacy.io/usage/spacy-101#annotations-pos-deps), `spaCy` figures out a lot of things about these tokens. For instance,

```
for token in doc:
    print(token.text, token.lemma_, token.pos_, token.tag_)
```

```
Apple Apple PROPN NNP
is be AUX VBZ
looking look VERB VBG
at at ADP IN
buying buy VERB VBG
U.K. U.K. PROPN NNP
startup startup VERB VBD
for for ADP IN
$ $ SYM $
1 1 NUM CD
billion billion NUM CD
```

It figured it out that `Apple` is a proper noun ("PROPN" and "NNP"; see here for the part of speech tags).

Let's read a book

Shall we load some serious book? You can use any books that you can find as a text file.

```
import urllib.request

book = urllib.request.urlopen('https://sherlock-holm.es/stories/plain-text/stud.t
```

```
# Convert bytes to string using UTF-8 decoding
book_str = book.decode('utf8')
type(book_str)  # This should return 'str'
```

```
str
```

```
book[:1000]
```

```
b'\n\n\n\n                                    A STUDY IN SCARLET\n\n
Arthur Conan Doyle\n\n\n\n\n\n\n\n                                    Table of
contents\n\n         Part I\n        Mr. Sherlock Holmes\n        The Science
Of Deduction\n        The Lauriston Garden Mystery\n        What John Rance
Had To Tell\n        Our Advertisement Brings A Visitor\n        Tobias
Gregson Shows What He Can Do\n        Light In The Darkness\n\n         Part
II\n        On The Great Alkali Plain\n        The Flower Of Utah\n
John Ferrier Talks With The Prophet\n        A Flight For Life\n        The
Avenging Angels\n        A Continuation Of The Reminiscences Of John Watson,
M.D.\n        The Conclusion\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n
PART I\n\n                    (Being a reprint from the reminiscences of\n
John H. Watson, M.D.,\n                    late of the Army Medical
Department.)\n\n\n\n\n\n          CHAPTER I\n          Mr. Sherlock
Holmes\n\n\n     In t'
```

Looks like we have successfully loaded the book. You'd probably want to remove the parts at the beginning and at the end that are not parts of the book if you are doing a serious analysis, but let's ignore them for now. Let's try to feed this directly into `spaCy`.

```
doc = nlp(book)
```

```
---------------------------------------------------------------------------
ExtraData                                 Traceback (most recent call last)
/tmp/ipython-input-16-1063915173.py in <cell line: 0>()
----> 1 doc = nlp(book)

                                 5 frames
/usr/local/lib/python3.11/dist-packages/srsly/msgpack/_unpacker.pyx in
srsly.msgpack._unpacker.unpackb()

ExtraData: unpack(b) received extra data.
```

Next steps: Explain error

```
doc = nlp(book_str)
```

```
# Get words that are not stopwords or punctuation
words = [token.text for token in doc if not token.is_stop and not token.is_punct]



from collections import Counter

# Count most common words
Counter(words).most_common(5)
```

```
[('\n     ', 3107),
 ('\n\n     ', 772),
 ('said', 207),
 ('man', 155),
 ('Holmes', 98)]
```

```
# Clean the book by removing extra spaces and line breaks
clean_text = ' '.join(book_str.split())

# Reprocess the cleaned text
doc_clean = nlp(clean_text)

# Rebuild the word list without stopwords or punctuation
clean_words = [token.text for token in doc_clean if not token.is_stop and not tok

# Recount and show the top 10 words
clean_counts = Counter(clean_words)
clean_counts.most_common(10)
```

```
[('said', 207),
 ('man', 155),
 ('Holmes', 98),
 ('little', 80),
 ('time', 77),
 ('way', 69),
 ('came', 67),
 ('face', 67),
 ('asked', 65),
 ('come', 64)]
```

```
# Save cleaned word counts for later use
word_cnt = Counter(clean_words)
```

```
import matplotlib.pyplot as plt
```

```
from wordcloud import WordCloud

# Create the word cloud object
wc = WordCloud(width=800, height=400, background_color='white')
wc.generate_from_frequencies(word_cnt)
```

```
<wordcloud.wordcloud.WordCloud at 0x79b02a9bc2d0>
```

```
# Plot the word cloud
plt.figure(figsize=(15, 7))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.show()
```

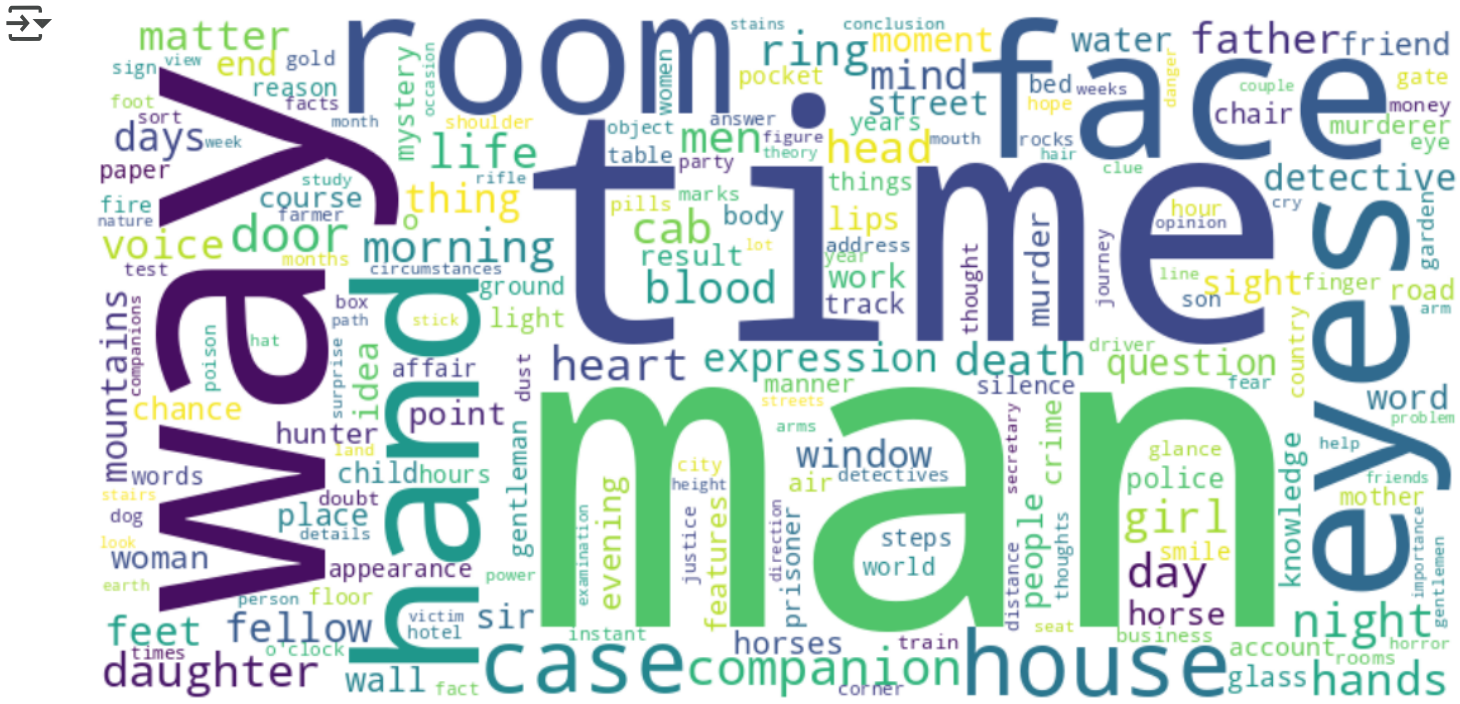


```
# Filter only NOUN tokens
nouns = [token.text for token in doc_clean if token.pos_ == 'NOUN' and not token.

# Count noun frequencies
noun_counts = Counter(nouns)

# Generate word cloud for nouns
wc_nouns = WordCloud(width=800, height=400, background_color='white')
wc_nouns.generate_from_frequencies(noun_counts)

# Plot it
plt.figure(figsize=(15, 7))
plt.imshow(wc_nouns, interpolation='bilinear')
plt.axis('off')
plt.show()
```



On encodings

What are we getting this error? What does it mean? It says `nlp` function expects `str` type but we passed `bytes` .

```
type(book)
```

```
bytes
```

Indeed, the type of `metamorphosis_book` is `bytes` . But as we have seen above, we can see the book contents right? What's going on?

Well, the problem is that a byte sequence is not yet a proper string until we know how to decode it. A string is an abstract object and we need to specify an encoding to write the string into a file. For instance, if I have a string of Korean characters like "안녕", there are several encodings that I can specify to write that into a file, and depending on the encoding that I choose, the byte sequences can be totally different from each other. This is a really important (and confusing) topic, but because it's beyond the scope of the course, I'll just link a nice post about encoding: http://kunststube.net/encoding/

```
"안녕".encode('utf8')
```

```
b'\xec\x95\x88\xeb\x85\x95'
```

```
# b'\xec\x95\x88\xeb\x85\x95'.decode('euc-kr') <- what happen if you do this?
b'\xec\x95\x88\xeb\x85\x95'.decode('utf8')
```

```
'안녕'
```

```
"안녕".encode('euc-kr')
```

```
b'\xbe\xc8\xb3\xe7'
```

```
b'\xbe\xc8\xb3\xe7'.decode('euc-kr')
```

```
'안녕'
```

You can decode with "wrong" encoding too.

```
b'\xbe\xc8\xb3\xe7'.decode('latin-1')
```

```
'¾È³ç'
```

As you can see the same string can be encoded into different byte sequences depending on the encoding. It's a really ~~annoying~~ fun topic and if you need to deal with text data, you must have a good understanding of it.

There is a lot of complexity in encoding. But for now, just remember that `utf-8` encoding is the most common encoding. It is also compatible with ASCII encoding as well. That means you can *decode* both ASCII and utf-8 documents with utf-8 encoding. So let's decode the byte sequence into a string.

```
# YOUR SOLUTION HERE
book_str = book.decode('utf8')
```

```
type(book_str)
```

```
str
```

Shall we try again?

```
doc = nlp(book_str)
```

```
words = [token.text for token in doc
         if token.is_stop != True and token.is_punct != True]
```

Let's count!

```
from collections import Counter

Counter(words).most_common(5)
```

```
[('\n     ', 3107),
 ('\n\n     ', 772),
 ('said', 207),
 ('man', 155),
 ('Holmes', 98)]
```

a lot of newline characters and multiple spaces. A quick and dirty way to remove them is split & join. The idea is that you split the document using `split()` and then join with a single space ` `. Can you implement it and print the 10 most common words?

```
# YOUR SOLUTION HERE
# Clean the book by removing extra spaces and line breaks
clean_text = ' '.join(book_str.split())

# Reprocess the cleaned text
doc_clean = nlp(clean_text)

# Rebuild the word list without stopwords or punctuation
clean_words = [token.text for token in doc_clean if not token.is_stop and not tok

# Recount and show the top 10 words
clean_counts = Counter(clean_words)
clean_counts.most_common(10)
```

```
[('said', 207),
 ('man', 155),
 ('Holmes', 98),
 ('little', 80),
 ('time', 77),
 ('way', 69),
 ('came', 67),
 ('face', 67),
 ('asked', 65),
 ('come', 64)]
```

Let's keep the object with word count.

```
word_cnt = Counter(clean_words)
```

Some wordclouds?

```
import matplotlib.pyplot as plt
```

Can you check out the `wordcloud` package documentation and create a word cloud from the word count object that we created from the book above and plot it?

```
# Implement: create a word cloud object

# YOUR SOLUTION HERE
from wordcloud import WordCloud

wc = WordCloud(width=800, height=400, background_color='white')
wc.generate_from_frequencies(word_cnt)
```

```
<wordcloud.wordcloud.WordCloud at 0x79b02c594890>
```

```
# Implement: plot the word cloud object

# YOUR SOLUTION HERE
plt.figure(figsize=(15, 7))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.show()
```



Q: Can you create a word cloud for a certain part of speech, such as nouns, verbs, proper nouns, etc. (pick one)?

```
# YOUR SOLUTION HERE
from collections import Counter
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# Extract only nouns from the document
```

```
noun_words = [token.text for token in doc if token.pos_ == 'NOUN' and not token.i

# Count the frequency of noun words
noun_counts = Counter(noun_words)

# Generate the word cloud
wc_nouns = WordCloud(width=800, height=400, background_color='white').generate_fr

# Plot the word cloud
plt.figure(figsize=(15, 7))
plt.imshow(wc_nouns, interpolation='bilinear')
plt.axis('off')
plt.show()
```



doc

```
"We have his cab," said Sherlock Holmes. "It will serve to take him
to Scotland Yard. And now, gentlemen," he continued, with a pleasant
smile, "we have reached the end of our little mystery. You are very
```

welcome to put any questions that you like to me now, and there is no danger that I will refuse to answer them."

PART II

The Country of the Saints.

CHAPTER I
On The Great Alkali Plain

In the central portion of the great North American Continent there lies an arid and repulsive desert, which for many a long year served as a barrier against the advance of civilisation. From the Sierra Nevada to Nebraska, and from the Yellowstone River in the north to the Colorado upon the south, is a region of desolation and silence. Nor is Nature always in one mood throughout this grim district. It comprises snow-capped and lofty mountains, and dark and gloomy valleys. There are swift-flowing rivers which dash through jagged cañons; and there are enormous plains, which in winter are white with snow, and in summer are grey with the saline alkali dust. They all preserve, however, the common characteristics of barrenness, inhospitality, and misery.

There are no inhabitants of this land of despair. A band of Pawnees or of Blackfeet may occasionally traverse it in order to reach other hunting-grounds, but the hardiest of the braves are glad to lose sight of those awesome plains, and to find themselves once more upon their prairies. The coyote skulks among the scrub, the buzzard flaps heavily through the air, and the clumsy grizzly bear lumbers through the dark ravines, and picks up such sustenance as it can amongst the rocks. These are the sole dwellers in the wilderness.

In the whole world there can be no more dreary view than that from the northern slope of the Sierra Blanco. As far as the eye can reach stretches the great flat plain-land, all dusted over with patches of alkali, and intersected by clumps of the dwarfish chaparral bushes. On the extreme verge of the horizon lie a long chain of mountain

```
peaks, with their rugged summits flecked with snow. In this great
stretch of country there is no sign of life, nor of anything
appertaining to life. There is no bird in the steel-blue heaven, no
movement upon the dull, grey earth--above all, there is absolute
silence. Listen as one may, there is no shadow of a sound in all that
mighty wilderness; nothing but silence--complete and heart-subduing
```

Topic modeling

Another basic text analysis is *topic modeling*. Imagine you have a bunch of documents and you want to know what are the main topics that are discussed in these documents. Topic modeling methods aim to extract these "topics" automatically based on the words that appear in the documents. In topic modeling, each topic can be thought of as a distribution over words. For instance, a topic can be "politics" and it can be represented as a probability distribution over words like "election", "vote", "president", etc.

Let's do a simple topic modeling with `bertopic` package. Although there is a long history of topic modeling, with many methods such as LDA, just like many other NLP tasks, topic modeling is also being replaced by LLM-based methods like BERTopic, so we'd like to try that as well. You should already have `scikit-learn` installed. Let's install `bertopic`.

```
pip install bertopic
```

Let's first get the dataset included in scikit-learn.

```
from sklearn.datasets import fetch_20newsgroups

newsgroup_data = fetch_20newsgroups(subset='all')
```

We can inspect what's inside. Feel free to play with it.

```
newsgroup_data.keys()
```

```
dict_keys(['data', 'filenames', 'target_names', 'target', 'DESCR'])
```

```
len(newsgroup_data.filenames)
```

```
18846
```

```
print(newsgroup_data.DESCR)
```

```
...                                          categories=categories)
>>> vectors_test = vectorizer.transform(newsgroups_test.data)
>>> pred = clf.predict(vectors_test)
>>> metrics.f1_score(pred, newsgroups_test.target, average='macro')
0.77310...

This classifier lost over a lot of its F-score, just because we removed
metadata that has little to do with topic classification.
It loses even more if we also strip this metadata from the training data:

  >>> newsgroups_train = fetch_20newsgroups(subset='train',
  ...                                       remove=('headers', 'footers',
  ...                                       categories=categories)
  >>> vectors = vectorizer.fit_transform(newsgroups_train.data)
  >>> clf = MultinomialNB(alpha=.01)
  >>> clf.fit(vectors, newsgroups_train.target)
  MultinomialNB(alpha=0.01, class_prior=None, fit_prior=True)

  >>> vectors_test = vectorizer.transform(newsgroups_test.data)
  >>> pred = clf.predict(vectors_test)
  >>> metrics.f1_score(newsgroups_test.target, pred, average='macro')
  0.76995...

  Some other classifiers cope better with this harder version of the task. T
  :ref:`sphx_glr_auto_examples_model_selection_plot_grid_search_text_feature
  example with and without the `remove` option to compare the results.

.. rubric:: Data Considerations

The Cleveland Indians is a major league baseball team based in Cleveland,
Ohio, USA. In December 2020, it was reported that "After several months of
discussion sparked by the death of George Floyd and a national reckoning ove
race and colonialism, the Cleveland Indians have decided to change their
name." Team owner Paul Dolan "did make it clear that the team will not make
its informal nickname -- the Tribe -- its new team name." "It's not going to
be a half-step away from the Indians," Dolan said."We will not have a Native
American-themed name."

https://www.mlb.com/news/cleveland-indians-team-name-change

.. rubric:: Recommendation

- When evaluating text classifiers on the 20 Newsgroups data, you
```

```
    should strip newsgroup-related metadata. In scikit-learn, you can do this
    by setting ``remove=('headers', 'footers', 'quotes')``. The F-score will b
    lower because it is more realistic.
- This text dataset contains data which may be inappropriate for certain NLP
    applications. An example is listed in the "Data Considerations" section
    above. The challenge with using current text datasets in NLP for tasks suc
    as sentence completion, clustering, and other applications is that text
    that is culturally biased and inflammatory will propagate biases. This
    should be taken into consideration when using the dataset, reviewing the
    output, and the bias should be documented.

.. rubric:: Examples

* :ref:`sphx_glr_auto_examples_model_selection_plot_grid_search_text_feature
* :ref:`sphx_glr_auto_examples_text_plot_document_classification_20newsgroup
* :ref:`sphx_glr_auto_examples_text_plot_hashing_vs_dict_vectorizer.py`
```

You can look into the data as well.

```
print(newsgroup_data.data[0])
```

```
From: Mamatha Devineni Ratnam <mr47+@andrew.cmu.edu>
Subject: Pens fans reactions
Organization: Post Office, Carnegie Mellon, Pittsburgh, PA
Lines: 12
NNTP-Posting-Host: po4.andrew.cmu.edu



I am sure some bashers of Pens fans are pretty confused about the lack
of any kind of posts about the recent Pens massacre of the Devils. Actually,
I am  bit puzzled too and a bit relieved. However, I am going to put an end
to non-PIttsburghers' relief with a bit of praise for the Pens. Man, they
are killing those Devils worse than I thought. Jagr just showed you why
he is much better than his regular season stats. He is also a lot
fo fun to watch in the playoffs. Bowman should let JAgr have a lot of
fun in the next couple of games since the Pens are going to beat the pulp out
regular season game.          PENS RULE!!!
```

You can load a more "clean" data like following.

```
docs = fetch_20newsgroups(subset='all',  remove=('headers', 'footers', 'quotes'))
```

```
print(docs[0])
```

```
I am sure some bashers of Pens fans are pretty confused about the lack
of any kind of posts about the recent Pens massacre of the Devils. Actually,
I am  bit puzzled too and a bit relieved. However, I am going to put an end
to non-PIttsburghers' relief with a bit of praise for the Pens. Man, they
are killing those Devils worse than I thought. Jagr just showed you why
he is much better than his regular season stats. He is also a lot
fo fun to watch in the playoffs. Bowman should let JAgr have a lot of
fun in the next couple of games since the Pens are going to beat the pulp out
regular season game.          PENS RULE!!!
```

BERTopic follows the convention of the scikit-learn API. You can fit the model with `fit_transform` method.

```
!pip install bertopic
```

```
Requirement already satisfied: tokenizers<0.22,>=0.21 in /usr/local/lib/pyth
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/py
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dis
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.
Downloading bertopic-0.17.3-py3-none-any.whl (153 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 153.0/153.0 kB 11.1 MB/s eta 0:0
Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl (3
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 363.4/363.4 MB 1.5 MB/s eta 0:00
Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.wh
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.8/13.8 MB 88.4 MB/s eta 0:00:
Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.wh
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.6/24.6 MB 75.9 MB/s eta 0:00:
Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 883.7/883.7 kB 41.3 MB/s eta 0:0
Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (66
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 664.8/664.8 MB 653.2 kB/s eta 0:0
Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl (21
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 211.5/211.5 MB 1.4 MB/s eta 0:00
```

```
Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 56.3/56.3 MB 9.6 MB/s eta 0:00:0
Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 127.9/127.9 MB 6.6 MB/s eta 0:00
Downloading nvidia_cusparse_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.wh
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 207.5/207.5 MB 6.0 MB/s eta 0:00
Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21.1/21.1 MB 84.8 MB/s eta 0:00:
Installing collected packages: nvidia-nvjitlink-cu12, nvidia-curand-cu12, nv
  Attempting uninstall: nvidia-nvjitlink-cu12
    Found existing installation: nvidia-nvjitlink-cu12 12.5.82
    Uninstalling nvidia-nvjitlink-cu12-12.5.82:
      Successfully uninstalled nvidia-nvjitlink-cu12-12.5.82
  Attempting uninstall: nvidia-curand-cu12
    Found existing installation: nvidia-curand-cu12 10.3.6.82
    Uninstalling nvidia-curand-cu12-10.3.6.82:
      Successfully uninstalled nvidia-curand-cu12-10.3.6.82
  Attempting uninstall: nvidia-cufft-cu12
    Found existing installation: nvidia-cufft-cu12 11.2.3.61
    Uninstalling nvidia-cufft-cu12-11.2.3.61:
      Successfully uninstalled nvidia-cufft-cu12-11.2.3.61
  Attempting uninstall: nvidia-cuda-runtime-cu12
    Found existing installation: nvidia-cuda-runtime-cu12 12.5.82
    Uninstalling nvidia-cuda-runtime-cu12-12.5.82:
      Successfully uninstalled nvidia-cuda-runtime-cu12-12.5.82
  Attempting uninstall: nvidia-cuda-nvrtc-cu12
    Found existing installation: nvidia-cuda-nvrtc-cu12 12.5.82
    Uninstalling nvidia-cuda-nvrtc-cu12-12.5.82:
      Successfully uninstalled nvidia-cuda-nvrtc-cu12-12.5.82
  Attempting uninstall: nvidia-cuda-cupti-cu12
    Found existing installation: nvidia-cuda-cupti-cu12 12.5.82
    Uninstalling nvidia-cuda-cupti-cu12-12.5.82:
      Successfully uninstalled nvidia-cuda-cupti-cu12-12.5.82
  Attempting uninstall: nvidia-cublas-cu12
    Found existing installation: nvidia-cublas-cu12 12.5.3.2
    Uninstalling nvidia-cublas-cu12-12.5.3.2:
      Successfully uninstalled nvidia-cublas-cu12-12.5.3.2
  Attempting uninstall: nvidia-cusparse-cu12
```

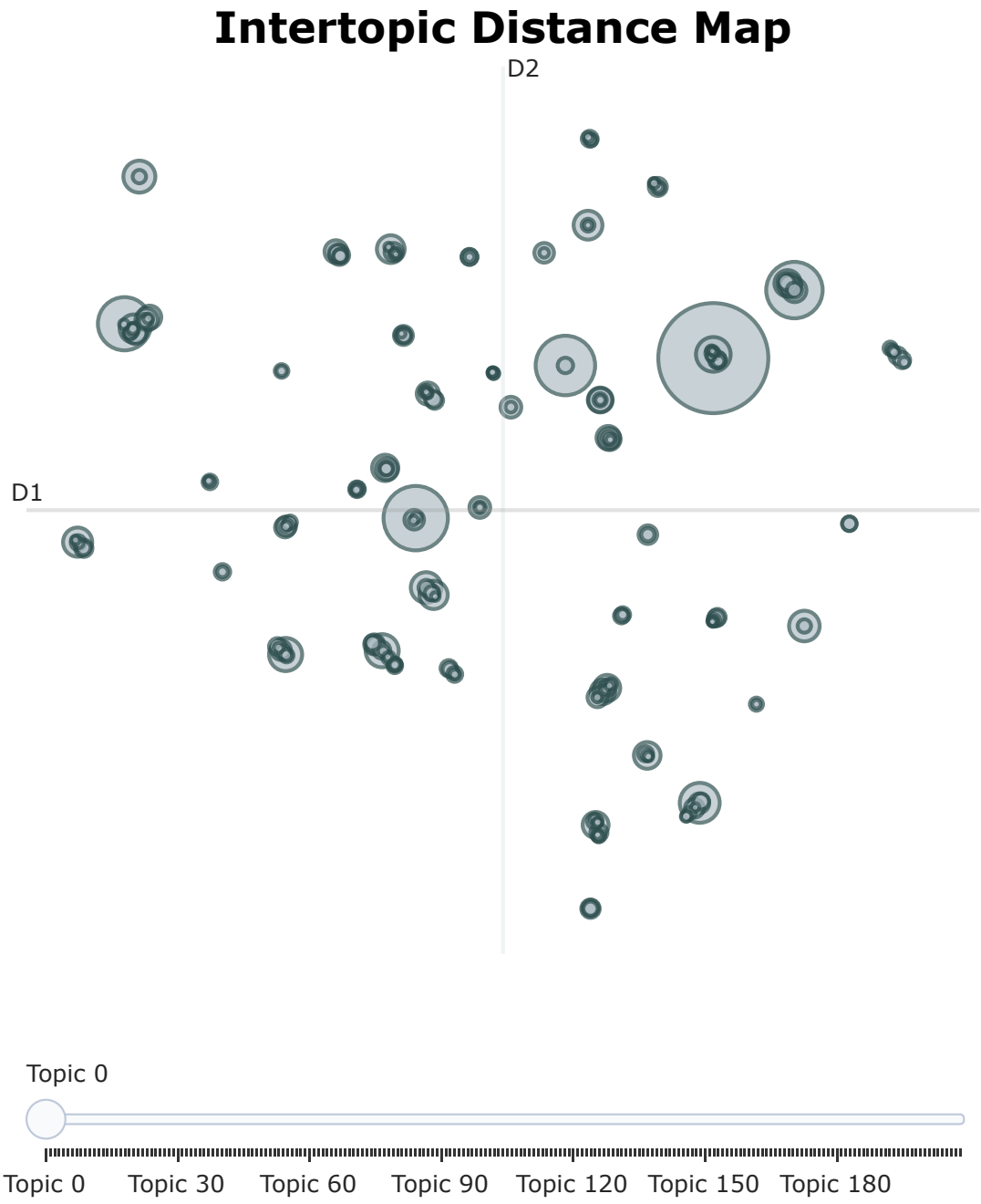
```
from bertopic import BERTopic

topic_model = BERTopic()
topics, probs = topic_model.fit_transform(docs)
```

```
modules.json: 100%                                                 349/349 [00:00<00:00, 19.6kB/s]
config_sentence_transformers.json: 100%                            116/116 [00:00<00:00, 10.5kB/s]
README.md:         10.5k/? [00:00<00:00, 798kB/s]
sentence_bert_config.json: 100%                                    53.0/53.0 [00:00<00:00, 3.82kB/s]
config.json: 100%                                                  612/612 [00:00<00:00, 56.9kB/s]
model.safetensors: 100%                                            90.9M/90.9M [00:01<00:00, 59.9MB/s]
tokenizer_config.json: 100%                                        350/350 [00:00<00:00, 25.0kB/s]
vocab.txt:         232k/? [00:00<00:00, 6.67MB/s]
tokenizer.json:         466k/? [00:00<00:00, 18.3MB/s]
special_tokens_map.json: 100%                                      112/112 [00:00<00:00, 9.42kB/s]
config.json: 100%                                                  190/190 [00:00<00:00, 15.0kB/s]
```

The package also has a quick visualizer, where you can see the topics and the words that are associated with the topics.

```
topic_model.visualize_topics()
```



Another visualization we can do (see https://maartengr.github.io/BERTopic/getting_started/visualization/visualization.html#visualize-documents) is to use the sentence embedding model to embed the documents into a vector space and visualize all the documents using the UMAP algorithm, along with the topics.

This time, I'll set the `min_topic_size` to 100 to reduce the number of topics it finds and to make the visualization simpler.

```
rom sentence_transformers import SentenceTransformer
rom umap import UMAP

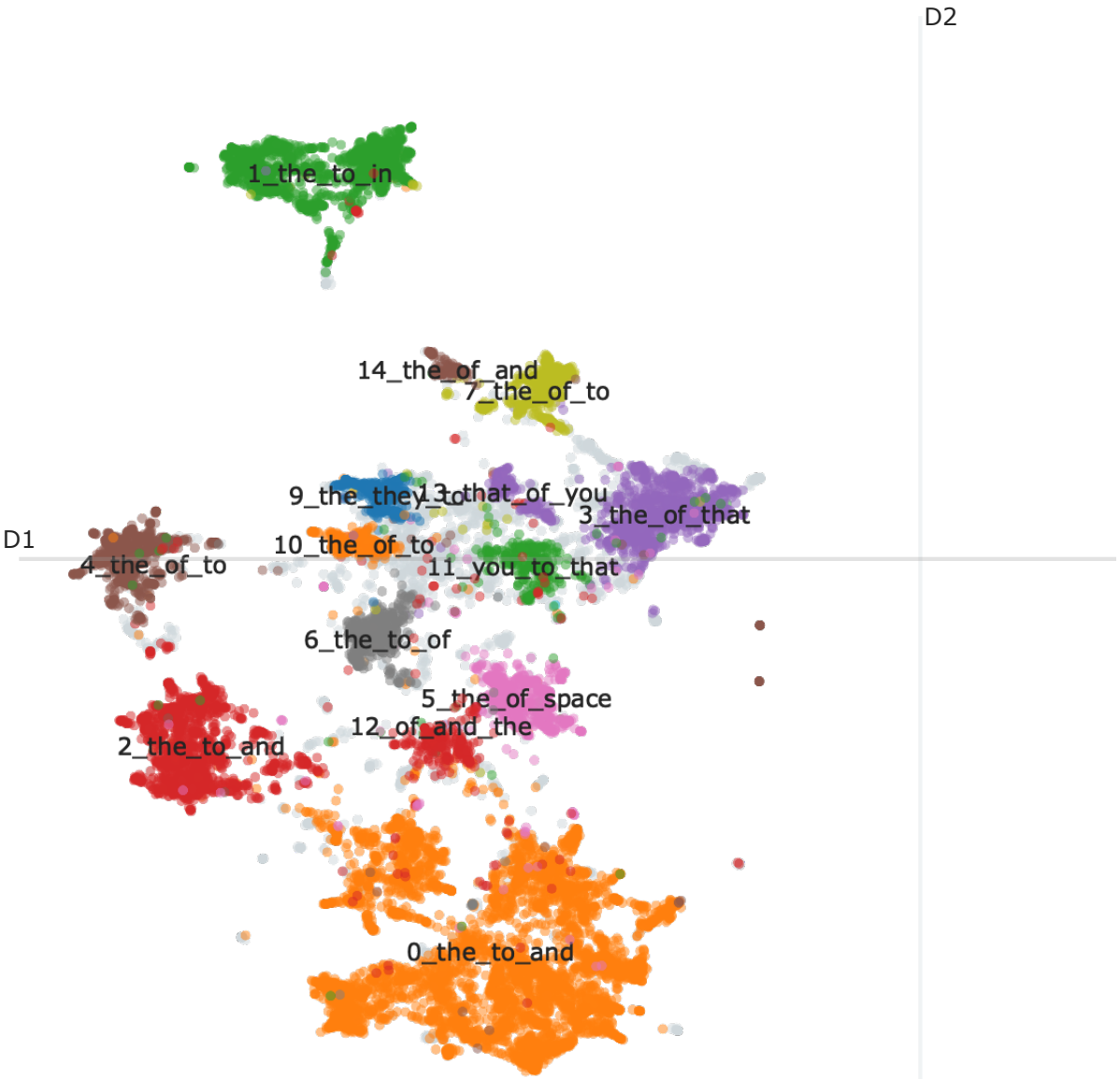
 Prepare embeddings
entence_model = SentenceTransformer("all-MiniLM-L6-v2")
mbeddings = sentence_model.encode(docs, show_progress_bar=False)

opic_model = BERTopic(min_topic_size=100).fit(docs, embeddings)

 Run the visualization with the original embeddings
opic_model.visualize_documents(docs, embeddings=embeddings)

 Reduce dimensionality of embeddings, this step is optional but much faster to perf
educed_embeddings = UMAP(n_neighbors=10, n_components=2, min_dist=0.0, metric='cosi
opic_model.visualize_documents(docs, reduced_embeddings=reduced_embeddings)
```

Documents and



Q: Can you identify an interesting dataset of documents and apply BERTopic & produce the visualization?

Answer

I used the `fetch_20newsgroups` dataset and applied BERTopic to find and visualize topics. I used sentence embeddings (`all-MiniLM-L6-v2`) and reduced them with UMAP for better visualization.

Two visualizations were created:

1. A topic summary (`visualize_topics()`)
2. A document scatterplot (`visualize_documents()`)

The results show meaningful topic clusters and how documents are grouped by topic.